

Problem A:

LLM-Assisted Netlist Exploration and Transformation

*Chung-Han Chou, Hung-Chun Chiu, Chih-Jen (Jacky) Hsu, and Yong Liu
(Cadence Design Systems, Inc.)*

Q&A

Q1. The examples in the document use named instances such as "and g1(w1, a, b);". In standard Verilog, unnamed instances are also valid, e.g., "and (w1, a, b);". Will all gate instances in the test benchmarks always have explicit instance names, or should we also handle unnamed instances?

A1. Yes, the test benchmarks will always have explicit instance names.

Q2. Section 3.2.a specifies 8 primitive gates and dff. Can we assume that the netlists will only contain Verilog built-in primitive keywords (and, or, nand, nor, not, buf, xor, xnor, dff), and not technology library cell names (e.g., NAND2X1, AOI22X1)?

A2. Yes, the netlists will only contain the built-in primitive keywords.

Q3. Will any sample test cases (input netlists + natural language request sequences) be released before the alpha submission deadline?

A3. Yes, we will release the sample test cases before the end of April.

Q4. I wonder which exact LLM models will be provided during the alpha, beta, and final testing phases. Could you please specify them?

Additionally, I would like to know if participants are permitted to fine-tune these models, or if the models can only be utilized via API as an agent or similar framework.

A4. ChatGPT 4o mini and Claude 4.5 Haiku will be provided. These models can only be used via API.

Q5. While reviewing the problem statement, I have encountered several technical questions regarding the evaluation environment and testcase specifications. I would greatly appreciate your clarification on the following points:

1. Executable Format

The problem statement specifies the executable naming convention (e.g., cada0001_alpha) but does not restrict the implementation language or packaging method. Is it acceptable to submit a self-contained executable packaged with tools such as PyInstaller, or must the submission be a natively compiled binary?

2. Network Access During Evaluation

Since the system is required to call an external LLM service (e.g., OpenAI or Anthropic API) during evaluation, will the TSRI Machine have outbound internet access to reach these cloud endpoints? This is critical to our architecture design.

3. Working Directory and File Path Convention

When the executable is invoked, what is the assumed working directory? Should all input Verilog files be expected relative to the working directory, and should output files (e.g., result netlists and log files) also be written relative to it?

4. DFF Module Definition in Input Netlists

Will the DFF module definition (as shown in Figure 2 of the problem statement) always be included in the input Verilog file, or should we assume the exact interface from Figure 2 and treat DFF instances as black boxes during parsing?

5. Availability of Sample Testcases

Are there any sample or example testcases available for download? Having concrete examples would help me better understand the expected input/output format and the range of task complexity. I would also like to ask whether additional testcases will be released before the final evaluation.

6. Validity Guarantee of Input Verilog Netlists

Will the input Verilog netlists provided during evaluation always be syntactically and semantically valid (i.e., free of undefined signals, unconnected ports, or non-standard

primitives)? Knowing whether defensive error handling for malformed inputs is necessary would significantly affect my parser design.

A5.

1. Submission of a PyInstaller package in a Docker image is acceptable.
2. Internet access is available only for the designated model API endpoints during evaluation; general internet access is not provided.
3. Yes. The Docker working directory is /app. Input files will be mounted under /app, and your program should read the design from the path given in the prompt and write all output files to the same testcase directory.
4. The DFF will appear as a primitive gate in the input Verilog file.
5. The test cases have been uploaded.
6. There may be floating or unconnected ports in the Verilog netlist.

Q6. We are a participating team for Problem A: LLM-Assisted Netlist Exploration and Transformation. As we are currently designing our agentic framework, we would like to seek clarification on several technical aspects to ensure our implementation is compatible with the official evaluation environment:

1. Compatibility with Model Context Protocol (MCP):

To enhance tool-use stability across different LLMs, we are considering using the Model Context Protocol (MCP). However, we have concerns regarding the Docker evaluation environment:

- a. Does the environment allow Inter-Process Communication (IPC) via stdio or local pipes between the Agent host and a tool server?
- b. Will the environment have internet access to install additional packages (e.g., the mcp Python SDK), or should we assume a strictly offline/pre-installed environment?
- c. If IPC is restricted or the environment is offline, an MCP-based architecture might fail. Could you provide guidance on whether MCP is supported or if we should strictly use standard Function Calling?

2. Scope of Required Functionalities:

The Problem A description provides several examples of tool functions. We would like to confirm whether the examples provided in the document constitute the exhaustive list of tools/APIs we need to implement, or if there will be additional, unlisted functions introduced during the final evaluation?

3. Evaluation Model Specifications:

Figure 6 of the configuration file mentions gpt-4o-mini and claude-haiku-4-5. Is it confirmed that only these two specific models will be used for the final evaluation? Or is there a possibility that other models (e.g., larger-scale models or different versions) will be utilized as well?

A6.

- 1a. IPC is acceptable, but the MCP server must reside on the local machine.
- 1b. Internet access is allowed only for model APIs; contestants should assume no general internet access for package installation.
- 1c. MCP is supported, but the environment is offline.
- 2. There will be hidden prompts in the final evaluation. Contestants should not only consider the tool functions in the document.
- 3. Only these two models will be used for evaluation. We will switch to different models only if both of them are deprecated.

Q7. Will the contest provide us the api keys for openai and anthropic? Or, do we have to use our own ai keys?

A7. Contestants should use their own API keys during development.

Q8. In Problem A, the “cost” and “cost_min” are mentioned, cost_min/cost decides our rank. What is “cost” exactly? It isn’t stated in the pdf.

A8. We will define the cost inside the prompt.

Q9. In Problem A, for optimization testcases, solutions are ranked by a cost ratio relative to cost_best.

Is the cost simply the objective stated in the request (e.g., minimize gate count), or is it computed differently?

A9. The cost will be defined inside the prompt.

Q10. EDA Engine: Must the EDA back-end be built entirely from scratch, or may we use existing tools (commercial or open-source) as the implementation — e.g.,

Cadence Genus/JasperGold via TCL scripting, or open-source tools such as Yosys or Icarus Verilog?

A10. We have no restrictions on EDA back-end tools.

Q11. LLM Evaluation: Will the final evaluation be run using one of the two specified LLMs (GPT-4o mini, Claude Haiku 4.5) or both? If both, how are the scores combined?

A11. We will consider each LLM as a separate case.

Q12. I have a few questions regarding the competition setup and APIs:

In the problem description, the dff module definition seems inconsistent with the dff used in the provided gate-level Verilog test examples. For example, in test35.v:

```
dff g10537(.RN(n1), .SN(1'b1), .CK(n0), .D(n49[2]), .Q(n194[2]));
```

However, the problem statement defines the dff module as:

```
module dff (  
    input wire clk,      // Clock input  
    input wire rst_n,    // Asynchronous active-low reset  
    input wire d,        // Data input  
    output reg q         // Data output  
);
```

The number of the ports do not appear to match.

A12. The dff provided in the released test case is the correct one. .SN is the active-low set signal.

Q13. Is the provided API fully compatible with the OpenAI API format and calling conventions? For example, does it support features such as Function Calling / Tool Calling in the same way as the OpenAI API?

A13. Yes, the provided OpenAI API key is fully compatible with the OpenAI API format. During development, contestants should use their own API keys. The contest

will replace the API key with its own during evaluation.

Q14. Since LLM responses are inherently non-deterministic, the same question may produce different answers with different wording while still having the same meaning. Therefore, We would like to clarify:

1. Will the evaluation be based on semantic equivalence rather than exact string matching?
2. Will the organizer use deterministic settings such as fixed random seeds or temperature = 0 during evaluation?
3. If an answer is semantically correct but phrased differently from the reference answer, will it still receive full credit?
4. Is there any public information about the evaluation metric or judging method (e.g., keyword matching, embedding similarity, LLM-as-a-judge, etc.)?

We would like to better understand the evaluation assumptions to ensure our implementation aligns with the official judging process.

A14.

1. Yes.
2. No.
3. Yes
4. The evaluation uses LLM-as-a-judge methodology based on semantic equivalence.

Q15. Are teams permitted to bundle statically-built third-party binaries (e.g., a standalone ABC build) inside the submission package and invoke them as subprocesses from the team executable? If yes, are there constraints on binary size, source-availability requirements, or license (must it be open-source / a permissive license)?

A15. Yes, third-party binaries are permitted with no size limit.

Q16. For "list all X" prompts where the answer can contain tens of thousands of items (e.g., test35 contains 19,682 NAND gates), is the grading pipeline expected to consume the full enumerated list, or is there a documented response-length / character cap above which the output is treated as malformed? If a cap exists, could you share the exact threshold so we can size our internal output behavior accordingly?

A16. Yes, full enumerated lists are expected. For large result sets, write the complete list to a file and provide the file path in your response.

Q17. What are the per-testcase (and per-run) RAM and disk limits in the evaluation environment? This determines whether we should default to a SAT-based equivalence checker vs. a BDD-based one, and how aggressively we may cache intermediate Boolean structures or AIG dumps.

A17. Please refer to the TSRI machine specifications.

Q18. For the GPT-4o-mini and Claude 4.5 Haiku endpoints provided via the evaluation config, are there per-request rate limits, per-testcase token budgets, or hard request timeouts we should design the agent to respect? Knowing the official policy will help us tune retry / back-off and fallback to rule-based routing safely.

A18. Rate limits follow the policies of GPT-4o-mini and Claude 4.5 Haiku. There is no token limit. The runtime limit is 60 seconds for each basic operation request (as defined in Section 4.1) and 300 seconds for all other requests.

Q19. Will an official cost estimator or evaluation tool be provided so that participants can locally validate their results and estimate their scores before submission?

A19. No

Q20. For the Final Submission evaluation, will the scoring be based on both public testcases and hidden testcases? If so, could you clarify how they will contribute to the final score?

A20. The final evaluation will include hidden test cases.

Q21. During implementation we have encountered several specification points where the intended interpretation is not obvious from the problem statement and the public

test cases. We would be very grateful for your clarification, so that our system can align with the official judging criteria.

1. Definition of "constant" inputs / signals

- Relevant questions:
 - testcase36 Q7 — "Report any AND gates with a constant 0 input in this design."
 - testcase32 Q17 — "Simplify the reported NAND gates by propagating their constant inputs."
 - testcase31 Q8 — "Is output n16 always 0 regardless of all inputs?"
 - Does "constant" refer only to pins structurally tied to 1'b0 / 1'b1 in the netlist, or does it also include signals that are functionally constant (provably 0/1 through upstream logic, e.g., via constant propagation or SAT)?
 - For testcase31 Q8 specifically: how should DFF initial state and unknown (X) values be treated when judging "always 0"?

2. Fan-in depth across sequential elements

- Relevant question pattern: "Which output bit has the deepest fan-in logic cone?"
- Should the fan-in cone terminate at DFF outputs (treating DFF.Q as a primary input, i.e., combinational depth only), or should it traverse through DFFs into prior cycles (sequential depth)?
- If traversal is expected, how should feedback loops and initial state be handled?

3. Path enumeration output size

- Relevant question pattern: "Provide a complete enumeration of paths between n0[1] and n63[0]." (For testcase14 this yields 289,366 paths; a literal listing exceeds several hundred MB.)
- Is "complete enumeration" expected to list every path literally, or is a path count plus representative samples acceptable?
- Is there a maximum response size or maximum number of paths the judge will accept? How is the boundary drawn between count-style questions and enumerate-style questions?

4. Judging criteria for transformation / rewrite questions

- Relevant questions:
 - testcase32 Q7 — "Try to replace all 2-input NAND gates that have one input tied to constant 1 with inverters. Ensure the design functionality does not change."
 - testcase32 Q17 — NAND constant propagation.
 - Is functional equivalence the sole judging metric, or are secondary metrics (gate count, area, logic depth, fan-out, etc.) also scored?
 - When multiple legal rewrites are possible (e.g., partial vs. full replacement, or different gate substitutions), which output is preferred?

5. Concurrent LLM API calls

- Are concurrent / asynchronous LLM API calls permitted during evaluation, i.e., may we issue multiple LLM requests in parallel for a single test case (or across test cases)?
- If yes, are there rate limits, per-question time budgets, or per-test total time budgets we should design around?

6. Multithreaded program design

- Independently of LLM calls, is our program itself allowed to be multithreaded / multi-process during evaluation (e.g., parallel netlist analysis, parallel SAT queries, worker pools)?
- Are there constraints on CPU cores, memory, or process count enforced by the judging environment?

A21.

1. A signal is functionally constant if it provably remains 0 or 1 for all possible inputs. The DFF initial state is 0, and X values are ignored.
2. Combinational depth only; treat DFF.Q outputs as primary inputs.
3. Complete enumeration means list every path literally. For very large result sets, write the output to file and provide the file path in the response.
4. Functional equivalence is the primary judging metric. Secondary metrics will be explicitly specified in the prompt if they are used for scoring. When multiple legal rewrites are possible, the prompt will clarify which optimization criteria to prioritize.
5. Only single-threaded execution is permitted, with one request at a time. Rate limits follow the policies of GPT-4o-mini and Claude 4.5 Haiku. There is no token limit. The runtime limit is 60 seconds for each basic operation request (as defined in Section 4.1) and 300 seconds for all other requests.

6. Only single-threaded execution is permitted. Please refer to the TSRI machine specifications for hardware constraints.

Q22. Regarding the 40 provided testcases, do the Verilog features appearing in these testcases already cover all possible Verilog constructs that may appear in the final evaluation benchmarks?

If there are additional Verilog features or syntax elements that do not appear in the released 40 testcases but may still appear during evaluation, could you please provide a list of those possible features?

A22. The Verilog syntax will follow the rules defined in Section 3.2 of the problem description.

Q23. How exactly is the cost calculated/evaluated during the contest?

A23. We will define the cost in the prompt.

Q24. Will participants be provided with either GPT-4o Mini or Claude Haiku 4.5 API credits?

A24. No, contestants should use their own API during development.